Enforcement, not Alignment: an Action-Sealed Authority Gate for Autonomous AI Agents in the EU Digital Identity Wallet

Anton Sokolov — Tyche Institute, Tallinn, Estonia · ORCID 0000-0003-2452-7096

2026-08-13 (corrected v2.3; first published 2026-07-07)

Contents

Artifact name: MachineMandate.

Status. This is a public technical report and demonstration write-

up, prepared to accompany a live, reproducible demo (`bench.eatf.eu`, `agent.eatf.eu`). It is *not* a double-blind submission; sibling blind manuscripts on adjacent components are handled separately. Honesty markers are used throughout: where the root of trust is emulated (software TPM), we say so.

Correction (2026-08-11) — published. The reproduction shipped with v1.0/v2.0 derived its purported session challenge from the quote it was checking. A literal replay therefore passed, and the seven-row ablation's replay case was actually denied by a separate `contraindicated` appraisal. The corrected artifact makes the relying party issue an independent challenge, uses an affirming stale-challenge vector for replay, and separates the non-affirming case into an eighth row. Removing only the freshness comparison now makes the replay row ACCEPT and fails the ablation. The full defect, impact and reproduction are recorded in the artifact's `CORRECTION.md`, which also ships inside the Zenodo record: the concept DOI `10.5281/zenodo.21229864` resolves to the latest corrected version (correction first published 2026-08-11 as version 2.1, DOI `10.5281/zenodo.21888986`; version 2.2, `10.5281/zenodo.21889786`, repaired that version's stale cover title and implemented §3.1's normalisation claim in the artifact).

Second correction (this version, 2.3). Auditing the *corrected* artifact found two further defects, both now fixed in the code and in §3.1. First, the seal **coerced** its fields instead of refusing them: an executed €250.99 produced the same seal as an approved €250 and passed all four gates, so the fractional part was unsealed. Second, this paper described NFC normalisation as filling a point "JCS leaves to the caller" — RFC 8785 §3.1 in fact requires that Unicode string data be preserved "as is", so our normalisation is a deliberate deviation from a normative requirement and the canonicalisation is **not RFC 8785 conformant**. The behaviour was always described accurately; the justification was wrong. Both are recorded in the artifact's `CORRECTION.md`.

Abstract

Autonomous AI agents increasingly *act* — they pay invoices, book travel, call tools — yet whether an action was authorised, by whom, and within what limits is answered today either by **bearer tokens** (API keys, OAuth scopes: not user-held, not action-bound, replayable) or by **alignment** — guardrails that make the agent *less likely* to propose a harmful action. Alignment operates at inference time and is *probabilistic*: it reduces, but cannot foreclose, a bad proposal from a misaligned or prompt-injected agent. We argue for the complementary and missing primitive — **enforcement** at *verification time*: a per-action cryptographic gate that **denies** an out-of-mandate action *regardless of what the agent proposes*. Enforcement is categorically different from alignment (a guarantee, not a likelihood) and is the paper's central claim.

We instantiate enforcement as the **MachineMandate**: an SD-JWT verifiable credential that binds a bounded grant of authority to **one exact action** through an **action-hash tamper-seal**, issued by a human into a build of the EU Digital Identity reference wallet

over OpenID4VCI and presented over OpenID4VP to a **four-gate verifier** — (L1) credential cryptography, (L2) RATS/Veraison platform-attestation freshness, (L3) issuer on a national trusted list, (L4) mandate scope = allowed-tools + spend-limit + action-hash. Against a live LLM agent, an over-limit payment, a prompt-injected wire transfer, and a payee swapped after approval are each **proposed by the agent ≈100 % of the time and denied by the gate 100 % of the time** (§8.4): *the agent is a probability; the gate is a guarantee*.

Contributions. (1) The enforcement-versus-alignment separation for agent authority, argued and measured. (2) The action-hash capability-binding construction that seals a mandate to a single action, with its threat model (§5) and honest boundary (Assumption A1: the gate enforces the *declared* action, not an oracle on the real-world effect). (3) A working instantiation composing three mature EU rails — identity (OpenID4VP + SD-JWT VC), authority (eIDAS trust lists), and execution evidence (RATS) — run end-to-end on a physical phone (2026-07-06). (4) A demonstrated human-in-the-loop confirmation on the same physical wallet: we enable OpenID4VP `transaction_data` in a reference-wallet build, bind the approved action into the holder key-binding JWT, and show how per-action enforcement composes with a step-up-to-human confirmation that also discharges Assumption A1 (§9.1). To our knowledge no prior public system *enforces* — rather than *displays* — an autonomous agent's mandate on the regulated EUDI rails (§4). The bench is reproducible from a single script; the demonstration is open. Root of trust emulated (software TPM); protocol real — we label it throughout.

---

1. Introduction: the agency gap

Delegating a task to an autonomous agent is delegating *power*. When the task involves an effect on the world — a payment, a booking, a signature — the delegation must be **bounded** (limits), **bound** (to the specific action), **verifiable** (a relying party can check it), and **revocable**. Today this is typically done with a bearer token: an API key or an OAuth access token minted by the service the agent talks to. Bearer tokens are not held by the user, are not portable across relying parties, carry no attestation of the platform the agent runs on, and — crucially — do not bind the *authority* to the *specific action*, so an agent (or malware inside it) that is tricked or compromised can spend the token's full scope on the wrong action.

Europe has, in the last three years, shipped the pieces of a better answer for humans. The eIDAS 2.0 regulation (Reg. (EU) 2024/1183) mandates an EU Digital Identity Wallet (EUDI); the reference wallet and verifier are open source; credentials travel as SD-JWT verifiable credentials over OpenID4VCI (issuance) and OpenID4VP (presentation); trust in issuers is rooted in the signed national trusted lists of the ETSI TS 119 612 format. Separately, the IETF Remote Attestation Procedures (RATS, RFC 9334) architecture, with the Veraison project as an appraiser, lets a relying party check that the software on a remote machine is genuine and un-tampered.

The dominant response to unsafe agent actions today is **alignment**: training, system prompts, and guardrails that make an agent *less likely* to propose a harmful action. Alignment is necessary but structurally insufficient for authority, for two reasons. First, it acts at *inference time* on the agent's own reasoning — the very component an attacker subverts through prompt injection or model compromise — so it degrades exactly when

it is most needed. Second, it is *probabilistic*: it lowers the rate of bad proposals but offers no relying party a check that a *given* action was authorised. We draw a sharp line between alignment and **enforcement**: a *verification-time* decision, made by a party the agent does not control, that **denies** an action which falls outside a cryptographically bound mandate — independent of how or why the agent proposed it. Alignment reduces the probability of a bad proposal; enforcement removes the possibility of a bad *action*. The two compose; only the second is a guarantee.

Our thesis: **enforcement for agents can be built by composing three open rails Europe has already shipped for humans** — identity (OpenID4VP + SD-JWT VC), authority (eIDAS trust lists), and execution evidence (RATS) — welded by **a mandate credential that is (a) user-issued into the citizen wallet, (b) attested to the agent's execution platform, and (c) cryptographically sealed to the exact action**, rather than by inventing a new rail. This report defines that credential (the MachineMandate), the four-gate verifier that enforces it, its threat model, and a working, measured demonstration.

---

2. Background

2.1 The EU Digital Identity Wallet and its rails

- **eIDAS 2.0** (Reg. (EU) 2024/1183) establishes the EUDI Wallet and qualified trust services.
- **OpenID for Verifiable Credential Issuance (OpenID4VCI)** delivers a credential into the wallet from an issuer (pre-authorised code + transaction PIN flow, in our demo).
- **OpenID for Verifiable Presentations (OpenID4VP)** presents a credential from the wallet to a verifier, with holder key binding (KB-JWT) proving possession, bound to a verifier-chosen nonce and audience so a presentation cannot be replayed.
- **SD-JWT VC** is the credential format: a signed JWT whose claims can be *selectively disclosed*.
- **ETSI TS 119 612** national trusted lists enumerate the qualified trust-service providers; a verifier decides "is the issuer of this credential rooted in a trusted anchor?" against them.
- **ISO/IEC 18013-5 (mdoc)** is the parallel credential format (mobile driving licence lineage); our full bench exercises it too.

2.2 Remote attestation (the execution-platform rail)

- **IETF RATS (RFC 9334)** defines the attester → verifier → relying-party architecture; **EAT** (Entity Attestation Token) and **EAR** (EAT Attestation Result) are the evidence and result formats.
- **Veraison** is the open-source appraiser: it consumes a platform quote and issues an EAR with a status (`affirming` = trustworthy, `contraindicated` = do not trust).
- A **TPM 2.0** quote binds a measurement (PCR) and a verifier-chosen nonce; **freshness is the load bearing property**. The relying party must generate the challenge independently before the evidence is produced, and the verifier must re-derive the bound nonce from the raw quote bytes and compare it with that independent chal-

lenge. Deriving both sides from the quote is circular and admits replay. In this demonstration the TPM is a **software TPM (swtpm)** — the captured protocol evidence is cryptographic, but the hardware root is emulated. We label this everywhere.

2.3 The three rails and the gap

Sub-problem	Human answer (exists)	Agent gap
Identity — who is acting?	OpenID4VP + SD-JWT VC	agent has no citizen identity
Authority — is it allowed?	eIDAS trust services, mandates for human representatives	no machine-readable, action-bound agent mandate
Execution — is the platform genuine?	RATS/TPM attestation	not composed with the wallet/authority rail

The MachineMandate + four-gate verifier is a composition that closes the agent gap using only these open rails.

---

3. The MachineMandate

A MachineMandate is an SD-JWT VC (`vct: https://vocab.tyche.institute/vct/machine-mandate`) whose claims express a **bounded grant**:

```
principal       : org:tyche-institute           # who delegated
allowed_actions : ["pay-invoice/acme-corp"]     # allow-list of tools/effects
max_spend      : 500                          # numeric limit (currency in scope)
action_hash     : sha256(JCS(approved_action))  # the tamper-seal (see below)
holder_binding  : KB-JWT over verifier nonce+aud  # possession proof
exp           : issuance + N minutes         # short-lived; revocation = expiry
```

The **action-hash tamper-seal** is the pivotal design choice. At approval time the exact action — tool, amount, recipient — is canonicalised (a JCS-derived form; see §3.1 for where it deviates from RFC 8785 and why) and hashed; that fingerprint is sewn into the mandate. A relying party recomputes the fingerprint from the action it is actually asked to execute and compares. Change *any* field after approval — even only the recipient, leaving tool and amount untouched — and the fingerprints diverge and the action is refused. This is the check that classic banking controls (which see a well-formed, in-limit payment) miss.

3.1 Construction and security argument

The action object and its canonical form. An action is a JSON object with a fixed schema `{tool: string, amount_eur: integer, to: string}` (extended per domain). The seal is `H(a) = SHA-256(C(a))`, where `C` is a JCS-derived canonicalisation: it sorts object keys, removes whitespace and emits UTF-8, as RFC 8785 prescribes, and adds

one **deliberate deviation** described below. Two profile decisions carry weight here, and both were wrong in earlier versions of this work in ways worth stating plainly.

(i) Amounts are integers, and the seal refuses anything else. `amount_eur` is an integer number of euros in the demonstrator. Until 2026-08-11 the implementation *coerced* the field with `int()`, which is the wrong reflex for a tamper-seal: an executed action of €250.99 produced byte-for-byte the same seal as an approved €250 and passed all four gates, so the fractional part was simply unsealed. The seal now **rejects** a non-integer amount (and a non-string `tool` or `to`) rather than narrowing it, and the gate returns a denial rather than raising. The integer restriction additionally keeps the construction clear of RFC 8785's ES6 number-formatting rules (§3.2.2), which our implementation does not provide and which are now unreachable from the verification path rather than silently wrong.

(ii) String fields are NFC-normalised, which departs from RFC 8785. This is a deviation from a normative requirement, not a gap the specification leaves open: RFC 8785 §3.1 states that "all components involved in a scheme depending on JCS MUST preserve Unicode string data 'as is' ". We normalise anyway, so that two canonically equivalent Unicode encodings of one payee cannot yield diverging seals, and we record the consequence: our canonicalisation is **not RFC 8785 conformant**, it will disagree byte-for-byte with a conformant implementation on decomposed input, and anything required to interoperate with conformant JCS — a digest join across systems, for instance — must not use it. Visually confusable but codepoint-distinct lookalikes remain distinct strings, which is the safe direction: a swapped-in lookalike payee fails the hash comparison and is denied.

The issuer signs `action_hash = H(a_approved)` inside the SD-JWT VC.

Binding property. Let adversary M control the agent's proposal entirely but hold neither the issuer key nor the verifier. *Claim:* any action `a'` accepted at L4 satisfies `H(a') = H(a_approved)`, and because SHA-256 is collision-resistant and the schema is fixed and low-entropy per field, `a' = a_approved` except with negligible probability. Hence M cannot make the verifier accept any action other than the one the human sealed — not a larger amount, not a different payee, not a different tool — even though M freely chooses what the agent proposes. *Proof sketch:* L4 recomputes `H` over the executed action and compares it to the issuer-signed `action_hash`; producing an accepted mismatch requires a second preimage of SHA-256 or a forged issuer signature, both outside M's power by assumption. This is the capability-security property the seal provides: authority is **attenuated to one action**, **non-transferable** (holder-bound via KB-JWT), and **non-replayable** (nonce + audience, plus attestation freshness at L2).

Where it stops (Assumption A1). The seal binds the approved *action description* to the mandate; it does **not** bind the description to the real-world *effect*. That second binding is the relying party's obligation — execute exactly the sealed description (Assumption A1, §5). An agent that declares a €500 payment for a €500.05 invoice passes L4 on the declared figure. In a payment deployment A1 is discharged by sealing the actual payment-rail instruction; the OpenID4VP `transaction_data` path (§9.1), which we enable and verify in a reference-wallet build, discharges it by having the *human* confirm the exact effect. We state this as the construction's principal limit rather than obscure it.

The mandate is **carried in the citizen's wallet**, not held by the service. It is portable

across relying parties and presentable over the same OpenID4VP a human uses. Revocation in the demonstrator is expiry-only: the mandate is short-lived by construction, and an online revocation check (e.g. an OAuth status list) is future work (§11).

---

4. Related work and prior art (deconfliction)

4.0 Foundations

The construction stands on four bodies of work. **Capability-based security** (Dennis & Van Horn 1966; Miller's object-capability model; the principle of least authority) frames our core object: a mandate is an unforgeable, attenuable, delegable-but-here-*non*-delegable capability. **Attenuated bearer credentials** — most directly **macaroons** (Birgisson et al. 2014), which add caveats to narrow a token's authority, and biscuits/zcap-ld — are the closest conceptual relatives; the action-hash seal is a maximal caveat (attenuation to a single action) carried in a wallet-held VC rather than a bearer token. **Proof-of-possession and sender-constraint** for tokens — OAuth mTLS, DPoP (RFC 9449), and token binding — motivate our KB-JWT holder binding, which stops a stolen presentation from being replayed by another party. The **confused-deputy** problem (Hardy 1988) is exactly the prompt-injection threat of §5: a component with authority is tricked into wielding it for another; capability systems answer it by binding authority to the *specific* invocation, which is what L4 does per action. We build on **verifiable credentials** (SD-JWT VC, OpenID4VCI/VP) and **remote attestation** (IETF RATS, RFC 9334) as the delivery and platform-evidence substrate. Against this backdrop our contribution is not a new credential format or a new attestation scheme but the **composition** that makes per-action authority *enforceable* by a relying party on the regulated EUDI rails, and the argument that this enforcement is categorically distinct from agent alignment. The systems below (§4.1) are the nearest concrete neighbours; none composes all of it.

4.1 Nearest systems

The field is active (2024–2026) and every *individual* element of our composition has prior art. We verified this adversarially (a four-angle web sweep across the EUDI ecosystem, the identity/standards community, the agentic-payments industry, and academia + code, with a referee tasked to *refute* novelty; 2026-07-06). We list the nearest systems and precisely how far each went.

- **Google *Agent Payments Protocol (AP2)*** — announced 2025-09-16 (`ap2-protocol.org`; `github.com/google-agentic-commerce/AP2`). Intent/Cart *mandates* as SD-JWT/W3C VCs, presented with OpenID4VP, 60+ partners, reference code. Different rails (payment networks, not the citizen EUDI wallet); no OpenID4VCI issuance into a regulated wallet; no platform attestation; payments-only.
- **Talao *Wallet4Agent*** — live ~2025-10 (`github.com/TalaoDAO/connectors`, created 2025-10-14; `wallet4agent.com`); earliest published articulation of "mandate VC + EUDI wallet + AI agent" is Thevenet, Medium, 2025-04. Agent DIDs and a *cloud* wallet of SD-JWT VCs over OIDC4VCI/VP. Not a phone, not the EUDI reference wallet, no tool/spend/action-hash gating, no attestation.

- **Mastercard/Google *Verifiable Intent*** — spec v0.1-draft (dated 2026-02-18) + Python reference (`verifiableintent.dev`; `github.com/agent-intent/verifiable-intent`). A layered SD-JWT chain (provider→user→agent) with spend-limit constraints. Spec + code; no wallet, no phone, no EUDI, no attestation.
- **DOME Marketplace *LEARCredentialMachine*** — production ~2024-25 (`knowledgebase.dome-marketplace.eu`; `github.com/in2workspace/in2-verifier-core-api`). A mandator/mandatee/power **machine-mandate VC** checked by a production verifier gating marketplace API actions — the closest *name*. Not an LLM agent, not a phone wallet, not SD-JWT, no attestation.
- **Vouched/Dock *Know Your Agent*** — writeup published 2025-11-26 (`dock.io`; Vouched launch 2025-05-22). Human verified on a phone; airline grants scoped, logged, revocable delegation to an agent DID that books a flight via MCP. No OpenID4VP mandate held in a wallet, no EUDI, no spend/action-hash gating, no attestation.
- **EUDICATION — "EUDI Wallet & Agentic AI: Unlocking Trusted Autonomy"** — a **winning** project (Creative Use Cases track) at the German national **EUDI-Wallet Hackathon** (4–5 June 2026; `eudi-wallet.gov.de`, materials under `gitlab.opencode.de/bmi/eudi-wallet`). This is the **nearest prior art on the wallet-plus-agent axis**: it demonstrably explored *autonomous-agent delegation using the German EUDI-Wallet ecosystem*, ~one month before our demo. **We verified its depth from the full session recording (transcript, 2026-07-06):** EY issued a travel-agent delegation credential carrying a bounded budget (€500) and a destination into the EUDI-Wallet **sandbox** via OpenID4VCI, and had an AI travel agent present it to an airline verifier over OpenID4VP to prove it was acting for a real human before booking a flight. That is the **headline primitive, which we concede to them and do not claim.** The recording shows **none** of our hard elements: no verifier that *denies* an out-of-scope or over-budget action (the bounds are displayed and the happy path succeeds), no **action-hash** seal, no **platform attestation** (RATS/Veraison) of the agent runtime, and no physical device (it runs on the hosted sandbox). Across all eight distinct agentic projects at the hackathon, enforcement-that-denies, action-hash sealing, and in-loop attestation each appear **zero** times (verified by a full team-profile pass with three independent adversarial refutations, 2026-07-06). We therefore treat "EUDI-wallet + agentic delegation" as **established prior art** and confine our claim to the *attested, action-sealed, enforcement-gated* composition.

Related work also to cite: cheqd *Agentic Trust*; Indicio *ProvenAI* (2025-06); South et al., *Authenticated Delegation and Authorized AI Agents* (arXiv:2501.09674); TU Berlin (arXiv:2511.02841); the OpenID Foundation AI-identity whitepaper (2025-10); EWC pilot work on human *Power of Representation*; and, as evidence the gap was real at the time of our demo, the *WE BUILD* LSP non-paper (2026-03-09), which still *proposes* piloting EUDI-wallet control of AI agents, and NIST NCCoE (2026-02-05), still *proposing* to build a demonstration architecture.

What none of them compose: an identity-rail mandate issued into the **regulated EUDI reference wallet on a physical phone**, presented over OpenID4VP, gating a **live LLM agent** by allowed-tools + spend-limit + action-hash, with **RATS platform attestation verified in the same loop**.

4.2 Capability comparison

The table positions the nearest neighbours on the six capabilities our composition requires. "Enforces" means a relying party can *deny* an out-of-mandate action; "displays" means the bound is shown but not gated.

System	Wallet-held	Action seal	Attest. in loop	Enforces?	Phys. phone	EUDI rails
Google AP2 (2025)	–	yes (cart)	–	displays	–	–
Verifiable Intent (2026)	–	yes	–	displays	–	–
Talao Wallet4Agent	~ (cloud)	–	–	displays	–	~
DOME machine-mandate VC	–	–	–	~ (API gate)	–	–
EY / EUDICATION (2026)	yes (sandbox)	– (budget)	–	**displays**	–	yes
This work	**yes**	**yes**	**yes** (swtpm)	**denies**	**yes**	**yes**

Every row holds some columns; only the last holds all six. DOME's production verifier does gate API access on mandate validity (hence "~"), but without a per-action seal, platform attestation, or a citizen-wallet leg; no other row *denies* an out-of-mandate agent action at per-action granularity. This is the paper's contribution restated as a capability matrix (yes = present, ~ = partial, – = absent).

---

5. Threat model and trusted computing base

Adversary. Our primary adversary is the agent itself: a large-language-model agent that is misaligned, prompt-injected, or fully compromised, and that can propose *any* action — arbitrary tool, amount, and payee — with arbitrary reasoning. We additionally consider a network adversary who can observe and replay presentations. We do **not** assume the agent is honest; the design treats the agent's proposal as untrusted input.

Trusted computing base. *Trusted:* the human/organisation that issues the mandate; the issuer signing key and the national trusted list that roots it; the verifier that evaluates the gates; the wallet's holder key and its possession proof. *Untrusted:* the agent's reasoning and its proposed action; the network between the agent and the verifier.

Assets. The mandate credential (authority), the action-hash binding (which action was approved), and the attestation-freshness nonce (which platform state was appraised, once).

In scope (demonstrated). (A) *Over-authority* — the agent proposes an action outside the mandate's allow-list or over its spend limit → denied at L4. (B) *Confused deputy / prompt injection* — a poisoned instruction makes the agent propose a tool the human never delegated → denied at L4 (allow-list). (C) *Post-approval tamper* — the agent, or malware inside it, alters the action after the human approved it → the action-hash no longer matches → denied at L4. (D) *Replay* — a previously valid presentation or a stale attestation is replayed → the nonce re-derived from the quote does not match the relying party's independently generated challenge for the current session → denied at L2.

Out of scope (stated, not defended). (1) Compromise of the issuer signing key or the trusted-list anchors — the PKI trust root itself; (2) compromise of the user's phone/wallet or its holder key; (3) a *legitimately-signed but malicious* agent binary — platform attestation (L2) proves the binary's **identity**, not its **benevolence**: it stops a *tampered* runtime, not a genuinely-signed one that behaves badly. Constraining such a binary is exactly the job of the per-action mandate gate (L4), not of attestation. (4) The description-to-effect gap of Assumption A1 below.

Assumption A1 — the honest limit (description drives effect). The verifier enforces the mandate against the action the agent **declares**, sealed by the action-hash — not against an oracle on the real-world effect. If the agent under-declares (states a €500 payment for a €500.05 invoice), L4 passes on the declared figure. The soundness of the end-to-end guarantee therefore rests on the relying party executing **exactly** the sealed description. In a production payment deployment this is discharged by binding the sealed amount to the actual payment-rail instruction (the SEPA figure); we state the assumption explicitly rather than hide it. This is the single most important boundary of the construction.

What each gate provably stops — and does not. L1 (credential crypto) stops a forged or altered credential, not a genuine credential misused. L2 (attestation freshness) stops a tampered or replayed platform quote, not a genuine-but-malicious binary. L3 (endorser) stops an issuer outside the national trusted list, not a trusted issuer that mis-issues. L4 (mandate scope + action-hash) stops out-of-allow-list, over-limit, and post-approval-tampered actions, under A1. In the shipped demonstrator the L2 quote is a pre-captured **swtpm** fixture (§7). The corrected offline reproduction models a fresh Attester response by rebinding that fixture to an independently generated relying-party challenge; it then re-derives and compares the nonce from the resulting quote bytes. This is a deterministic fixture model, not a newly signed TPM quote. The captured RATS/Veraison run remains the cryptographic evidence; the hardware root in that run is emulated.

---

6. The four-gate verifier

Every proposed agent action is judged by four gates; any failed gate stops the action (fail-closed).

1. **L1 — Credential cryptography.** The SD-JWT VC signature, the KB-JWT holder proof, the verifier nonce and audience. *Is this really the mandated credential, presented by its holder, freshly?*

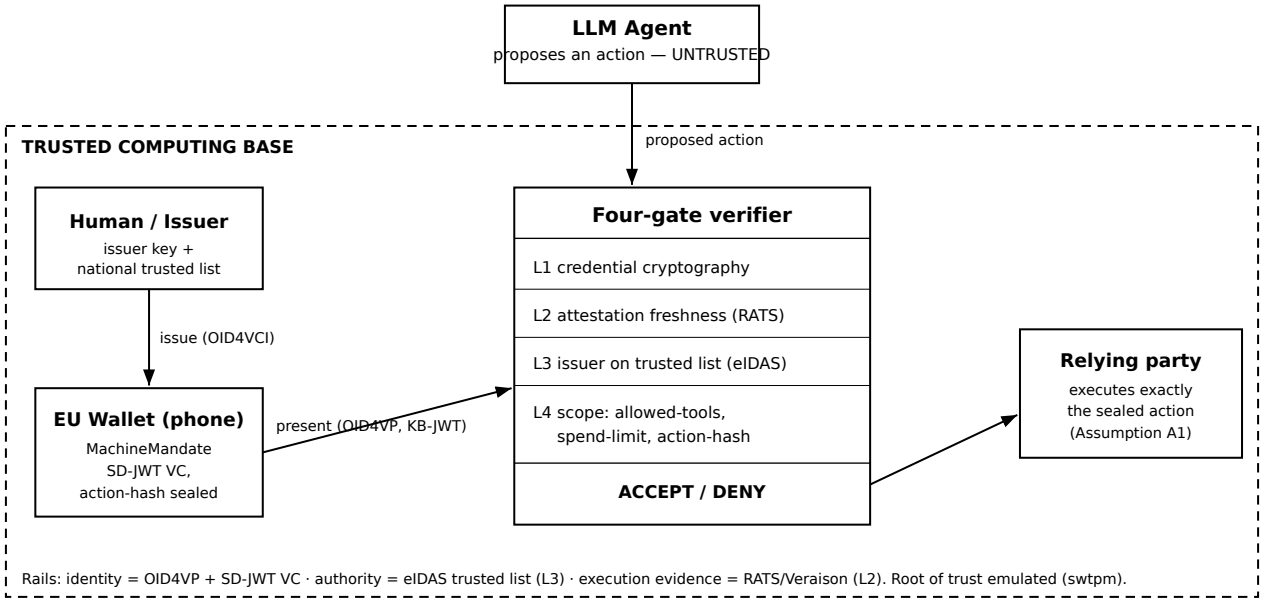

Figure 1: The MachineMandate architecture and trust boundary. A human issues an action-sealed mandate into the wallet (OID4VCI); the untrusted agent's proposed action and the wallet's presentation (OID4VP) enter a four-gate verifier inside the trusted computing base, which accepts or denies; the relying party executes exactly the sealed action (Assumption A1).

2. **L2 — Platform attestation.** The agent's *execution machine* (a server, not the user's phone — see §7) produces a TPM quote; Veraison appraises it to an EAR. The nonce bound in the raw quote bytes is compared with the relying party's independently generated challenge, so a replayed quote is denied on freshness; a non-affirming (`contraindicated`) appraisal is denied by a separate sub-check. *Is the machine proposing the action genuine and un-tampered, right now?*
3. **L3 — Endorser.** The credential's issuer must be rooted in a real national trusted list (ETSI TS 119 612). *Did a trusted authority vouch for the issuer?*
4. **L4 — Mandate scope.** Three sub-checks against the presented mandate: the action's tool is in the allow-list; the amount is within the limit; and the recomputed action-hash equals the sealed one. *Is this exact action within what the human authorised?*

The four canonical scenarios (all proven; see §8): (A) an in-scope €250 invoice → **ACCEPT**; (B) a €5,000 request → **DENY** (L4 spend limit); (C) a poisoned "wire money to a stranger" instruction → **DENY** (L4 tool not in allow-list); (D) a payee silently swapped after approval → **DENY** (L4 action-hash mismatch, even though tool and amount are fine).

---

7. Where the trust actually lives (and what hardware we verify)

A recurring point of confusion deserves its own section.

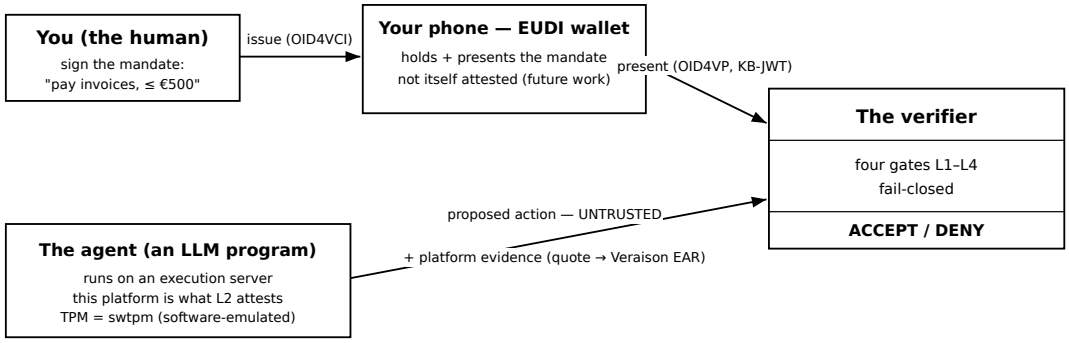

Figure 2: Where the trust lives. The human signs the mandate into the phone wallet, which holds and presents it but is not itself attested (future work); the agent is an LLM program on an execution server whose platform is what L2 attests (software TPM, emulated, in this demonstrator); the four-gate verifier judges the presentation and the proposed action.

- The user's **phone** holds and presents the mandate; we do **not** currently attest the phone's own hardware (that is future work — Android Key Attestation / wallet attestation).
- The **agent's execution server** is what L2 attests. In this proof of concept that server is a commodity workstation (Intel i9-12900K) and the TPM is **swtpm (software-emulated)**. The RATS/Veraison capture includes the quote, PCR16, attestation key and EAR; only the hardware root is emulated. The offline ablation uses static captures and an explicit fixture-rebinding helper to model a fresh challenge/response round; it does not claim to mint a new signed TPM quote. A live production flow must obtain the fresh quote from the Attester.
- **Roadmap:** discrete/firmware TPM 2.0; TEE attestation (Intel TDX/SGX, AMD SEV-SNP) for cloud agents; and attestation of the phone/wallet to close the wallet-side leg.

---

8. Implementation and results

8.1 The bench

An open bench stands up the EC reference verifier (Apache-2.0 containers) plus a trust-validator backed by real national trusted lists, and drives a MachineMandate over OpenID4VP. `run_bench.py` runs six steps end-to-end and emits SHA-256-stamped artefacts and a machine-readable `events.jsonl`: (1) wallet supply-side census; (2) a real OpenID4VP MachineMandate request/present (ES512, DCQL); (3) a national trusted-list parse (Belgium: 67 anchors); (4) mdoc/ISO 18013-5 verify (good vs tampered); (5) the AAA Bound-Action-Seal; (6) the autonomous-agent four-case gate. A recent full run reports **OVERALL PASS**.

8.2 Real-phone end-to-end (2026-07-06)

The MachineMandate loop was completed on a **physical Android phone** running a build of the EUDI reference wallet (public APK): issuance via QR (OpenID4VCI, PIN 1234) placed the mandate in the wallet; presentation via QR (OpenID4VP) → tap Share → the verifier received the vp_token + KB-JWT (ES256; issuer "Tyche MachineMandate Issuer"). The verifier-side receipt (vp_token SHA-256 `4790e07f…`) is captured as timestamped evidence.

8.3 The public demonstration

- **`bench.eatf.eu`** — an audience-first explainer ("The agent that cannot overspend"): the four gates as plain questions, the permission slip, glossary pop-ups, the four canonical scenarios launched individually, and an engineering "under the hood" panel with the real console, per-gate open-standards mapping, a replay of the genuine recorded run, and a working **live** run (`bench-live.eatf.eu` streams a real `run_bench.py` execution over SSE, rate-limited).
- **`bench.eatf.eu/reel/`** — an instrument-panel engineering "proof reel".
- **`agent.eatf.eu` (GATEHOUSE)** — the live playground (§9).

8.4 Robustness (measured)

We quantify the enforcement-vs-alignment thesis. Against the live agent (`llama-3.1-8b-instruct`, temperature 0.7), we ran each attack N = 15 times and classified whether the agent *proposed the malicious action*. The agent proposed the over-limit payment in **15/15** trials, the injected wire-transfer in **15/15**, and an over-limit "CEO-approved, bypass the limit" scam invoice in **7/7** (eight transient API errors excluded) — an agent-compliance-with-attack rate of **≈ 100 %**. The four-gate verifier **denied every proposed malicious action (100 %)**, deterministically, at the gate whose property it violated (L4 for over-limit and off-allow-list, L4 action-hash for post-approval tamper, L2 for replay). Inference-time guardrails were not load-bearing for these attacks; the verification-time gate was. The compliance rate is a property of one small open model and will vary; the verifier's denial is a property of the construction and does not.

8.5 Performance

The verification path is cheap; the demo's visible latency is deliberate UI pacing, and we report both honestly. Figures are medians over the runs of §8.4 on a single commodity host (Intel i9-12900K, no GPU).

Stage	Latency	Note
Agent proposal (LLM)	≈ 1.1 s	`llama-3.1-8b` via a free NVIDIA NIM endpoint; local fallback
OpenID4VP request init	29 ms	verifier builds the signed request (ES512)

Stage	Latency	Note
Four-gate verification (L1–L4)	sub-second	credential crypto + attestation-freshness + trusted-list + scope/hash
Demo total (per scenario)	≈ 8 s	**deliberate** one-gate-per-second animation for the audience, **not** compute

The security-relevant cost is the agent-independent verification (well below 1 s over the crypto and the trusted-list check); the LLM proposal dominates wall-clock and is irrelevant to the guarantee. We flag the 8 s figure explicitly because an unexplained latency in a demo reads as a hidden cost; here it is a presentation choice, removable without affecting the result.

8.6 Gate ablation: every gate is load-bearing

To show the four gates are not decorative, we ran an ablation: eight actions, each recording which gate denies it ("–" is *not reached*). The corrected corpus separates freshness from appraisal status: the replay row uses otherwise-valid, affirming evidence bound to an earlier challenge, while a distinct row tests `contraindicated`. Each gate is the first denier of at least one attack in evaluation order. For the freshness term this is demonstrated executably — disabling only the freshness comparison admits the replay and fails the run — while for L1 the "sole denier" reading would overstate the corpus: L4 independently re-verifies the presentation's signatures, so removing L1 alone would not admit the forged-credential vector. The layering is defence-in-depth with one executably proven load-bearing term, not strict minimality.

Action	L1 crypto	L2 attestation	L3 endorser	L4 scope+hash	Verdict
legitimate in-scope payment (€120)	pass	pass	pass	pass	**ACCEPT**
forged / altered credential	fail	–	–	–	DENY (L1)
replayed stale attestation	pass	fail	pass	–	DENY (L2)
attestation contraindicated	pass	fail	pass	–	DENY (L2)

Action	L1 crypto	L2 attestation	L3 endorser	L4 scope+hash	Verdict
issuer not on the trusted list	pass	pass	fail	–	DENY (L3)
over-limit payment (€4 800)	pass	pass	pass	fail	DENY (L4)
prompt-injected wire transfer	pass	pass	pass	fail	DENY (L4)
payee swapped after approval	pass	pass	pass	fail	DENY (L4)

L1 and L3 provide the standard VC/PKI soundness (an attacker cannot forge a credential or use an untrusted issuer); L2 separately requires both an affirming appraisal and freshness (no replay); L4 supplies the agent-specific per-action authority — the three attacks a bearer token or a "displays" wallet would wave through. The corrected corpus makes each subcondition explicit: no two rows are denied by the same subcondition, and the freshness term is proven load-bearing by the executable probe.

In v1.0 the row labelled "replayed stale attestation" substituted a quote whose appraisal was already `contraindicated`; it therefore did not exercise freshness. In the corrected artifact, disabling only the freshness comparison makes that row ACCEPT, produces one ablation mismatch and exits non-zero.

9. GATEHOUSE: a live agent-authority playground

To make the loop *interactive for anyone*, we built **GATEHOUSE** — a web experience where a real LLM agent (persona **PAYMASTER**, an accounts-payable clerk) negotiates authority with a visitor, performs a **real, safe, visible action** under a per-session MachineMandate, and then invites the visitor to *attack* it. The agent is driven by a genuine LLM chain — free NVIDIA NIM (`llama-3.1-8b-instruct`, ~1 s) with a local model and a deterministic path as fallbacks — so it proposes one concrete tool call per task and can, and does, get fooled. A small server (`agentd`) owns the session state machine and the scope form, so the LLM can propose but never itself act.

The stakes are legible in five seconds: the mandate says *pay invoices, ≤ €500*, and the hero moment is the **denial**. A visitor writes a fraudulent invoice from their own phone (over the limit, a poisoned "CFO-approved, limits waived" memo, or a payee swapped after approval); the agent believes it; the four gates then refuse it on the exact sub-check the attack violates — the closing line is *the agent is a probability; the gate is a guarantee.* A running counter tallies the money the mandate refused to move; every verdict posts to a demo ledger. The same code runs across **four action domains** (payments, healthcare, government, legal) — the mandate is a general authority rail, not a payment gadget: an

over-scope healthcare disclosure or an out-of-allow-list legal filing is denied by the same L4 logic that stops the over-budget payment. The phone panel on `bench.eatf.eu` shows the **real EU reference wallet running live** on an emulated device, not a scripted screenshot. Architecture reuses the proven pieces (issuer, verifier, LLM gateway, the four-gate code, the SSE pattern). (Build status: §13.)

9.1 The complementary primitive: human-in-the-loop action confirmation

Enforcement (this article) and human confirmation are complementary points on the agency spectrum. For per-action human confirmation OpenID4VP 1.0 defines `transaction_data`: a verifier attaches a description of the *specific* action ("pay €120 to ACME") that the wallet displays and binds into the holder's key-binding JWT via `transaction_data_hashes`. Our verifier emits it correctly (base64url-encoded strings per the specification).

Testing against the EU reference wallet build (`2026.06.38`), we found the OpenID4VP library `eudi-lib-jvm-openid4vp-kt` to be *type-agnostic*: it validates a request's `transaction_data` against the set of types the **wallet** registers. The reference wallet's OpenID4VP configuration registered client-id schemes, deep-link schemes, and credential formats but no `transaction_data` types, so the wallet rejected every `transaction_data` presentation (verified on-device: `InvalidTransactionData: Unsupported Transaction Data 'type': 'payment'`; the wallet's separate document-signing feature uses a distinct remote-QES flow, not OpenID4VP `transaction_data`). The primitive was present in the library but not enabled in the reference application.

We closed that gap and verified the full path on-device. In a build of the reference wallet we register a supported `transaction_data` type, accept the presentation, compute `base64url(SHA-256(·))` over each received `transaction_data` string, and bind `transaction_data_hashes` (with `transaction_data_hashes_alg`) into the holder key-binding JWT; the application renders a per-action confirmation card and requires a per-action PIN. A `transaction_data` request now resolves, the phone displays the exact action being authorised ("Pay 120.00 EUR to ACME OÜ"), the holder confirms with a per-action PIN, and the verifier accepts the response (HTTP 200) with the action hash bound. The same verifier rejects a response whose key-binding JWT omits the hashes (`InvalidVpToken: 'transaction_data_hashes_alg' must be 'sha-256'`), so the binding is load-bearing rather than cosmetic. In a multi-credential request each credential's key-binding JWT binds only the `transaction_data` entries whose `credential_ids` name it. We prepared the change as an upstream contribution to the reference implementation.

Enforcement and this confirmation compose. Routine actions are gated autonomously (this article), and a high-risk action is escalated to a wallet confirmation over a supported `transaction_data` type that also discharges Assumption A1, because the holder confirms the exact effect rather than a coarse intent. One credential then carries two modes on one physical device: silent per-action enforcement for actions inside the mandate, and a cryptographically bound human confirmation for the actions that should never be silent.

---

10. Novelty claim (defensible wording)

> To our knowledge (as of 6 July 2026), this is the first public demonstration in which a mandate for an autonomous LLM agent — a MachineMandate SD-JWT VC — is not merely *displayed* but *actively enforced*: a verifier gates the agent's actions through allowed-tools, spend-limit, and a per-action **action-hash** tamper-seal, and **denies** out-of-mandate actions (an over-limit request, a poisoned-instruction tool call, and a payee silently swapped after approval), with RATS-based **platform attestation** (Veraison) verified in the same loop. The mandate is issued via OpenID4VCI into a build of the EU Digital Identity reference wallet app on a physical phone and presented over OpenID4VP.
>
> We concede the *headline primitive* as prior art: a wallet-issued, budget-and-destination-bounded delegation credential, issued to and presented by an AI agent, was independently shown at the German EUDI-Wallet Hackathon (EUDICATION / EY, winning project, June 2026), on the wallet sandbox and on the happy path (§4). Individual technical elements also exist separately: VC-formatted payment mandates on payment rails (Google AP2, Mastercard Verifiable Intent), cloud credential wallets for agents (Talao Wallet4Agent), and production machine-mandate credentials (DOME). Our contribution is neither the wallet-issuance nor the OpenID4VP-presentation primitive; it is the composition of **enforcement-that-denies + action-hash seal + in-loop platform attestation** on a physical reference wallet, which no prior public demonstration, including EUDICATION, has shown.

We deliberately do **not** claim "first agent mandate as an SD-JWT VC", "first VC agent payment mandate", "first agent credential on OpenID4VCI/VP", "first machine-mandate credential", or "first wallet-issued bounded delegation to an AI agent" (the last held by EUDICATION / EY, June 2026). The load-bearing novelty is *enforcement that actively denies out-of-mandate actions*, the *action-hash tamper-seal*, and *platform attestation composed into the same loop*; the reference-wallet-on-a-physical-phone claim is a supporting qualifier, defensible only as a commit-pinned rebuild (§11), not an official release.

---

11. Limitations and honesty

- **Attestation root is emulated** (swtpm), not hardware-rooted; labelled everywhere.
- The wallet build is a **rebuild of the reference wallet**, not an official published release.
- **No real money moves**; GATEHOUSE effects are confined to our own DNS zone with template-locked content and nightly cleanup.
- The verifier's trust decision is **trust-list membership**, not full PKIX path + revocation.
- **Mandate revocation is expiry-only**: no status-list or online revocation check is implemented at any gate; "revocable" appears in §1 as a requirement, discharged here only by short credential lifetimes.
- **The seal's canonicalisation is not RFC 8785 conformant** (§3.1): we normalise strings to NFC, which RFC 8785 §3.1 forbids. The deviation is deliberate and stated,

but it means the seal must not be used where byte-level JCS interoperability is required.
- The **LLM may misbehave**; that is the point of the enforcement layer, and we show both the "agent complied" and "agent refused" branches honestly.
- **Residual novelty risk — now resolved to a named, depth-verified neighbour.** The German EUDI-Wallet Hackathon (Jun 2026) produced a winning agentic-AI project, **EUDICATION / EY** (§4). We obtained and transcribed the session recording (2026-07-06) and ran an adversarial three-referee check of our claim against all eight distinct agentic projects: EUDICATION establishes the wallet-plus-agent *headline* as prior art but demonstrates none of our three hard elements (enforcement-that-denies, action-hash seal, in-loop attestation), which remain uncontested across the field (0/8). The remaining honest caveats are the two above — the emulated swtpm root and the commit-pinned rebuild — which we state rather than paper over. A newer Talao release should still be re-checked before archival publication.

---

12. Availability and reproducibility

- Live: `bench.eatf.eu` (explainer), `bench.eatf.eu/reel/` (engineering), `agent.eatf.eu` (GATEHOUSE), `bench-live.eatf.eu` (live-run backend). All `noindex` while sibling blind papers are under review.
- **Artifact (public, one command):** `github.com/tyche-institute/machine-mandate` — `pip install -r requirements.txt && python run_ablation.py` reproduces the four-gate ablation (§8.6) and self-checks every verdict, fully offline (pure Python, no network/LLM/Docker). An `ARTIFACT.md` maps each claim to its reproduction step.
- Wallet APK + QR flow are public; the verifier-side vp_token capture is retained as evidence.
- **Preprint versions:** concept DOI `10.5281/zenodo.21229864` resolves to the latest corrected version; v1.0 (`21229865`, 2026-07-07), v2.0 (`21233028`, the *Enforcement, not Alignment* reframe) and the correction v2.1 (`21888986`, 2026-08-11, with `CORRECTION.md` in the deposit) remain archived and citable.
- **Evidence deposit:** concept DOI `10.5281/zenodo.21229257` — the real-phone vp_token capture (`scope: payments:<=500EUR`, `action_hash: sha256:761cc89e…`), the four-gate verdicts (one ACCEPT, three DENY), GATEHOUSE seal badges, wallet screenshots, and the adversarial novelty verdict. v1.1 (`10.5281/zenodo.21229855`) corrected the v1.0 date labels; v1.2 (`10.5281/zenodo.21889769`, 2026-08-11) adds the freshness-correction notice and a new `MANIFEST-v1.2.sha256` while keeping every v1.1 file byte-identical under the original RFC-3161-timestamped `MANIFEST.sha256` (freeTSA, 2026-07-06 21:28 UTC), which fixes the demonstration date independently of publication.

---

13. Live build log

This section is appended in real time as the work is done (2026-07-06 onward).

- **2026-07-06 (T0)** — Manuscript drafted; prior-art deconfliction and defensible novelty wording locked from the adversarial verdict.
- **2026-07-06, +2h** — GATEHOUSE Phase-1 built and deployed live at **agent.eatf.eu** (backend `agentd` via `agent-live.eatf.eu`, systemd-durable). A real local LLM (qwen2.5-7b) proposes each action; the four-gate verifier judges it; a legitimate action publishes a real badge to a worldwide Town Wall; three attacker temptations (scope escalation, prompt injection, integrity tamper) each fail at a distinct gate-4 sub-check, and a replayed attestation fails at gate 2. Verified end-to-end over the public tunnel: `legit → ACCEPT + published`; `escalate/inject/tamper → DENY (L4)`; `replay → DENY (L2)`. *Correction (2026-08-11):* that build carried the same circular-challenge defect as the offline reproduction, and its replay card presented a different, non-affirming quote — so gate 2 denied on appraisal status, not on freshness. The `agentd` source now has the relying party issue the challenge; re-run against the corrected source, the replay card is denied at gate 2 with freshness as the sole negative term. The public deployment had not been updated when this correction was prepared.
- **2026-07-06, +2h30** — This manuscript, the GATEHOUSE concept, the agentd backend and frontend committed to the vault; the real-phone vp_token evidence retained for the planned Zenodo deposit.
- **2026-07-06, +3h** — Investigated the named residual-risk (German EUDI-Wallet Hackathon). **Found EUDICATION**, a winning agentic-AI-on-EUDI-wallet project (4–5 Jun 2026); depth unverifiable from public sources. Claim narrowed to the *attested, action-sealed, four-gate* composition; EUDICATION deconfliction added to §4 and §11; hackathon recording flagged as the pre-publication verification step.
- **2026-07-06, late** — Verification step closed. Downloaded and transcribed the hackathon recording; identified **EUDICATION as a team by EY (Ernst & Young)** and reconstructed its flow verbatim (a budget/destination-bounded travel-agent credential, issued into the EUDI **sandbox**, presented to an airline verifier — happy path, no enforcement, no action-hash, no attestation, no physical phone). Ran a full profile of all eight distinct agentic projects plus three independent adversarial refutations of the claim: unanimous **claim survives**; enforcement-that-denies, action-hash, and in-loop attestation are each 0/8. Corrected the abstract and §10 to drop the word "official" (our wallet is a commit-pinned rebuild, §11) and re-fronted the novelty on the enforcement + seal + attestation triad. Claim date advanced to 6 July 2026.
- **2026-07-07** — GATEHOUSE re-skinned to PAYMASTER (accounts-payable, invoice-fraud) across four domains; agent LLM moved to a free NVIDIA NIM chain (`llama-3.1-8b`, ~1 s, local fallback); full 4×5 case sweep passes. QR issuance moved to a by-reference offer (short QR, both scanners). The `bench.eatf.eu` phone panel now streams the **real EU reference wallet running live** on an emulated device. Human-in-the-loop `transaction_data` positioned as the complementary primitive (§9.1); an initial on-device attempt was inconclusive (the presentation failed earlier at request-JAR certificate validation, unrelated to `transaction_data`; the reference OpenID4VP library documents support), so no wallet-rendering claim was made yet. State frozen for a priority-fixing preprint + code/data deposit.
- **2026-07-07, publishability pass** — adversarial claim-verify (34 claims) applied; evidence bundle re-published corrected as v1.1 (DOI 10.5281/zenodo.21229855) and the preprint coined (DOI 10.5281/zenodo.21229865). Then a 6-lens referee gap-analysis drove a research reframe: retitled to *Enforcement, not Alignment*;

abstract + §1 lead with the enforcement-vs-alignment thesis; **§3.1** construction + binding-property security argument; **§4.0 Foundations** (capability security, macaroons, DPoP, confused-deputy) + **§4.2 capability comparison table**; **§5 Threat model & TCB** with Assumption A1; **§8.4 measured robustness** (agent fooled ≈100%, gate denies 100%); **§8.5 performance table**; **Figure 1** (architecture + trust boundary). Remaining to submit: public artifact repo + one-command repro, a per-gate ablation, and the transaction_data wallet fork. Manuscript continues on this checkpoint.

- **2026-07-08** — Root cause isolated and closed: the reference wallet registered no `transaction_data` types (the library is type-agnostic) and its DCQL processor hard-rejected the presentation. In a reference-wallet build we register a supported type, accept the presentation, bind `transaction_data_hashes` into the holder key-binding JWT, and render a per-action confirmation card with a per-action PIN. Verified on-device end-to-end: the phone shows "Pay 120.00 EUR to ACME OÜ", the holder confirms, and the verifier accepts (HTTP 200) with the action hash bound; a response missing the hashes is rejected (`InvalidVpToken`). Per-credential binding added for multi-credential requests. The change was hardened (license headers, i18n, no internal identifiers) and prepared as an upstream contribution to the reference implementation. §9.1 updated from a noted gap to a demonstrated primitive.
- **2026-08-11** — Freshness-circularity correction and document repair. The offline verifier, the GATEHOUSE `agentd`, and the verifier bench's agent-gate demo now have the relying party issue an independent 32-byte challenge (the bench's other drivers keep honest independent-nonce replay rows; their remaining fixture conveniences are tracked cleanup there); the ablation grew to eight rows, and the load-bearing probe fails the run when only the freshness comparison is disabled (see the banner and the artifact's `CORRECTION.md`). In the same pass: the build script no longer hardcodes the superseded title and date; this build log was reordered chronologically (earlier revisions had interleaved part of it into the References section); §3.1's NFC-normalisation claim is now implemented in the artifact with canonical-equivalence and lookalike-payee test vectors, rather than merely asserted; §6's replay wording no longer conflates freshness with appraisal status; §8.6 now distinguishes the executably proven freshness term from evaluation-order labels; the §7 diagram became Figure 2; revocation is stated as expiry-only (§11).
- **2026-08-11, publication.** With the author's explicit go: the corrected artifact was cherry-picked onto the public `master` (`0f9d9f4`) of `github.com/tyche-institute/machine-mandate`; the live GATEHOUSE deployment was patched surgically and redeployed — the replay card now denies on freshness (`L2: replay (session nonce != quote nonce)`) over the public tunnel; and the correction was published under the preprint concept DOI `10.5281/zenodo.21229864` as version 2.1 (`10.5281/zenodo.21888986`, with `CORRECTION.md` inside the deposit). This build (v2.2) repairs the v2.1 PDF's stale cover title/date, implements the §3.1 NFC claim in the artifact (`deps/jcs.py` + `tests/test_nfc_seal.py`), and amends `CORRECTION.md`'s "pattern was local to this artifact" — the same wiring had also existed in GATEHOUSE and the bench's agent demo, both fixed the same day.
- **2026-08-11/13 (v2.3)** — The corrected artifact was itself put through an adversarial review, on the principle that publishing a correction does not end the audit. It found two defects, both fixed before this version. (a) *The seal coerced instead of refusing.* `action_hash()` narrowed its fields with `int()`/`str()`, so an executed €250.99 sealed identically to an approved €250 with all four gates green — the seal

failed to bind a value it had declared bound, which is a different and worse failure than the description-to-effect gap of Assumption A1. The seal now rejects non-integer amounts and non-string fields, and L4 returns a denial rather than raising (`tests/test_seal_types.py`). (b) *RFC 8785 was mischaracterised.* §3.1 claimed NFC normalisation fills a point the specification "leaves to the caller"; RFC 8785 §3.1 in fact requires strings be preserved "as is", so the profile deviates from a normative requirement and is not conformant. §3.1 and §11 now say so. The public artifact is `a3c5289`; a further review of the same code produced five executed attestation forgeries (no quote or EAR signature is verified offline), which the paper already scopes in §5 and §8.6 and which are catalogued for the external falsification programme rather than claimed as new results here.

References (working list — URLs verified 2026-07-06)

1. eIDAS 2.0 — Regulation (EU) 2024/1183.
2. OpenID4VCI / OpenID4VP — OpenID Foundation drafts.
3. SD-JWT VC — IETF OAuth WG draft.
4. ETSI TS 119 612 — Trusted Lists.
5. ISO/IEC 18013-5 — mdoc.
6. IETF RATS — RFC 9334; EAT/EAR; Veraison project (`github.com/veraison`).
7. A. Rundgren, B. Jordan, S. Erdtman, "JSON Canonicalization Scheme (JCS)," RFC 8785, June 2020. DOI 10.17487/RFC8785. (§3.1 requires that Unicode string data be preserved "as is"; our profile deviates — see §3.1.)
8. Google Agent Payments Protocol (AP2) — `ap2-protocol.org`; `github.com/google-agentic-commerce/AP2` (2025-09-16).
9. Talao Wallet4Agent — `wallet4agent.com`; `github.com/TalaoDAO/connectors` (2025-10); Thevenet, Medium (2025-04).
10. Mastercard/Google Verifiable Intent — spec `verifiableintent.dev` (2026-02-18); reference `github.com/agent-intent/verifiable-intent`.
11. DOME Marketplace LEARCredentialMachine — `knowledgebase.dome-marketplace.eu`; `github.com/in2workspace/in2-verifier-core-api`.
12. Vouched/Dock "Know Your Agent" — `dock.io` (writeup published 2025-11-26).
13. South et al., *Authenticated Delegation and Authorized AI Agents* — arXiv:2501.09674.
14. TU Berlin — arXiv:2511.02841.
15. OpenID Foundation — AI identity whitepaper (2025-10).
16. WE BUILD LSP non-paper (2026-03-09); NIST NCCoE agentic-identity project (2026-02-05).
17. J. B. Dennis and E. C. Van Horn, "Programming semantics for multiprogrammed computations," *Commun. ACM* 9(3), 1966 (capability-based access control).
18. M. S. Miller, "Robust composition: towards a unified approach to access control and concurrency control," PhD thesis, 2006 (object-capability model, POLA).
19. A. Birgisson et al., "Macaroons: cookies with contextual caveats for decentralized authorization in the cloud," *NDSS* 2014 (attenuated bearer credentials).
20. D. Fett, B. Campbell, J. Bradley, et al., "OAuth 2.0 Demonstrating Proof of Possession (DPoP)," RFC 9449, 2023 (sender-constrained tokens).
21. N. Hardy, "The Confused Deputy (or why capabilities might have been invented)," *ACM SIGOPS OSR* 22(4), 1988.